## 目标

1）并行与并发

2）线程的定义

3）线程的创建方式

4）线程的生命周期

5）线程的启动和停止

6）线程的通信

7）线程安全问题

8）synchronized 使用方式

9）synchronized 锁状态及锁升级

10）volatile 的作用

11）happens-before 模型

# 1 线程基础

## 1.1 课前知识：

硬件：CPU、内存、磁盘、网络，硬件才是承载程序执行的基础。

软件：最大化的利用硬件资源。

方式：线程数量、JVM 内存分配大小、网络通信机制（BIO、NIO、AIO）、磁盘 IO

## 1.2 并发与并行

### 1.2.1 并行

并行表示在同一个时间点,有多个任务都在进行当中。并行是需要多个 CPU 才可以完成的。如果说每一个 CPU 都在运行一个程序,那么 4 个 CPU 就可以让 4 个程序“并行”运算。

咕泡出品 必属精品

### 1.2.2 并发

一个并发程序是具备处理多个任务的能力。并发并不需要有多个 CPU,单个 CPU 通过时间片的方式,不同时间片处理不同任务,可以让程序“看起来”是都在执行的。

## 1.3 什么是线程

线程：程序中的一个顺序控制流程，也是 CPU 中的最小调度单位。

线程与 CPU 核心数的关系。如果线程数小于或者等于 CPU 核心数的时候，则 CPU 能同时运行，如果，大于的时候，CPU 只能通过上下文切换的方式轮转交替执行线程。

### 1.3.1 守护线程和本地线程

Java 中的线程分为两种：守护线程（Daemon）和用户线程（User）。任何线程都可以设置为守护线程和用户线程，通过方法 Thread.setDaemon(bool on)；true 则把该线程设置为守护线程，反之则为用户线程。Thread.setDaemon()必须在 Thread.start()之前调用，否则运行时会抛出异常。

**两者的区别：**
唯一的区别是判断虚拟机(JVM)何时离开，Daemon 是为其他线程提供服务，如果全部的 User Thread 已经撤离，Daemon 没有可服务的线程，JVM 撤离。也可以理解为守护线程是 JVM 自动创建的线程（但不一定），用户线程是程序创建的线程；比如 JVM 的垃圾回收线程是一个守护线程，当所有线程已经撤离，不再产生垃圾，守护线程自然就没事可干了，当垃圾回收线程是 Java 虚拟机上仅剩的线程时，Java 虚拟机会自动离开。

## 1.4 线程的创建方式

### 1.4.1 Thread

new Thread().start()

### 1.4.2 Runnable

new Thread(new Runnable()).start()

### 1.4.3 Callable/Future

new Exectors.newFixedThreadPool(1).submit(new Callable())

## 1.5 线程的生命周期

new：初始化状态

runnable：运行状态

咕泡出品必属精品

（运行状态 runnable 分为：start：就绪状态和 running：运行中）

timed_waiting：限期等待
Thread.sleep(long),Obj.wait(long),LockSupport.parkUntil(long)

waiting：无限期等待 wait(),LockSupport.park()

blocked:阻塞状态，在用锁的情况下（synchronized），没有获取到锁的线程进入 blocked 状态。

terminated：终止状态，执行完 run 方法之后，自动被回收。

### 1.5.1 查看状态指令：

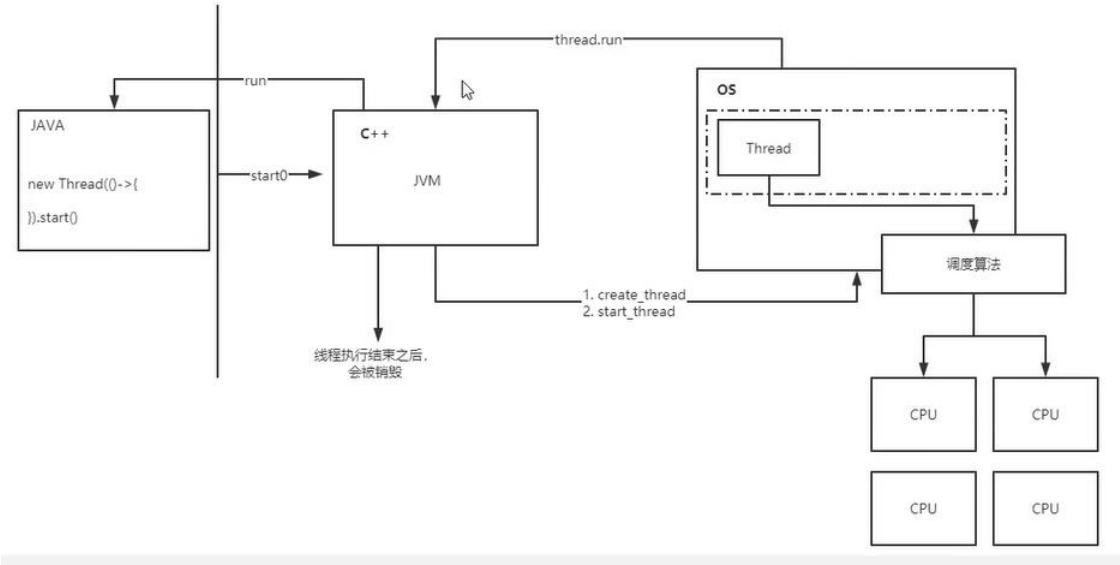
1.jps 指令查看我当前的进程 ID

2.jstack 线程 ID

## 1.6 线程的启动和停止

### 1.6.1 启动：

调用 start 方法启动，调用 run 方法只是普通的方法调用。




### 1.6.2 终止：

run 方法执行完了之后自动终止

必属精品

Thread.stop（不建议使用，对程序不友好，如果线程任务正在执行的话，调用 stop 会强制是的线程中断

Thread.interrupt(),这个是友好的中断。

#### 1.6.2.1 interrupt()的作用：

interrupt()的作用，如果 sleep，wait 是不是马上会中止（会，底层 C++会调用 unpark 唤醒线程，在 java 中会捕获 interruptException 异常，然后等待线程自己处理）

1.设置一个共享变量值

2.唤醒处于阻塞下的线程

源码举例

```
public class InterruptDemo02 implements Runnable{
    @Override
    public void run() {
        while(!Thread.currentThread().isInterrupted()){ //false
            try {
                TimeUnit.SECONDS.sleep(20);
                System.out.println("1231");
            } catch (InterruptedException e) {
                e.printStackTrace();
                System.out.println(Thread.currentThread().isInterrupted
());
                //可以不做处理。
                //继续中断 ->
                Thread.currentThread().interrupt(); //再次中断
                //抛出异常。。
            }
           // Thread.currentThread().interrupted();  //再次复位
            System.out.println(Thread.currentThread().isInterrupted());
        }
        System.out.println("processor End");
    }
    public static void main(String[] args) throws InterruptedException
{
        Thread t1=new Thread(new InterruptDemo02());
        t1.start();
        TimeUnit.SECONDS.sleep(5);
      //  Thread.sleep(1000);
        t1.interrupt(); //有作用 true
    }
}
```

咕泡出品必属精品

**JAVA 中如何停止一个线程？interrupt()、interrupted()、isInterrupted()的作用以及区别在哪？**

1. stop,过时了，因为会强行中止一个线程，不会等到线程业务执行，会导致结果不可控

2. 优雅的结束，interrupt()，会抛出 InterruptedException，线程自己决定要不要结束，异常会触发复位

   interrupt()：更改线程的中断标记，唤醒处于阻塞状态下的线程。

   interrupted()：获取线程的中断标记，并且执行复位操作

   isInterrupted()：获取线程的中断标记，但是不会进行复位操作

   native：native 是一个计算机函数，一个 Native Method 就是一个 Java 调用非 Java 代码的接口。方法的实现由非 Java 语言实现，比如 C 或 C++。

# 2 并发挑战

## 2.1 通过代码问题引入

```
public class TestDemo {
    public static int count=0;
    public static void add(){
        try {
            Thread.sleep(1);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        count++;
    }

    public static void main(String[] args) {
        for(int i=0;i<1000;i++){
            new Thread(()->TestDemo.add()).start();
        }
        try {
            Thread.sleep(3000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("运行结果："+count);
    }
}
```

运行结果是一个小于 1000 的随机数

咸鱼出品 必属精品

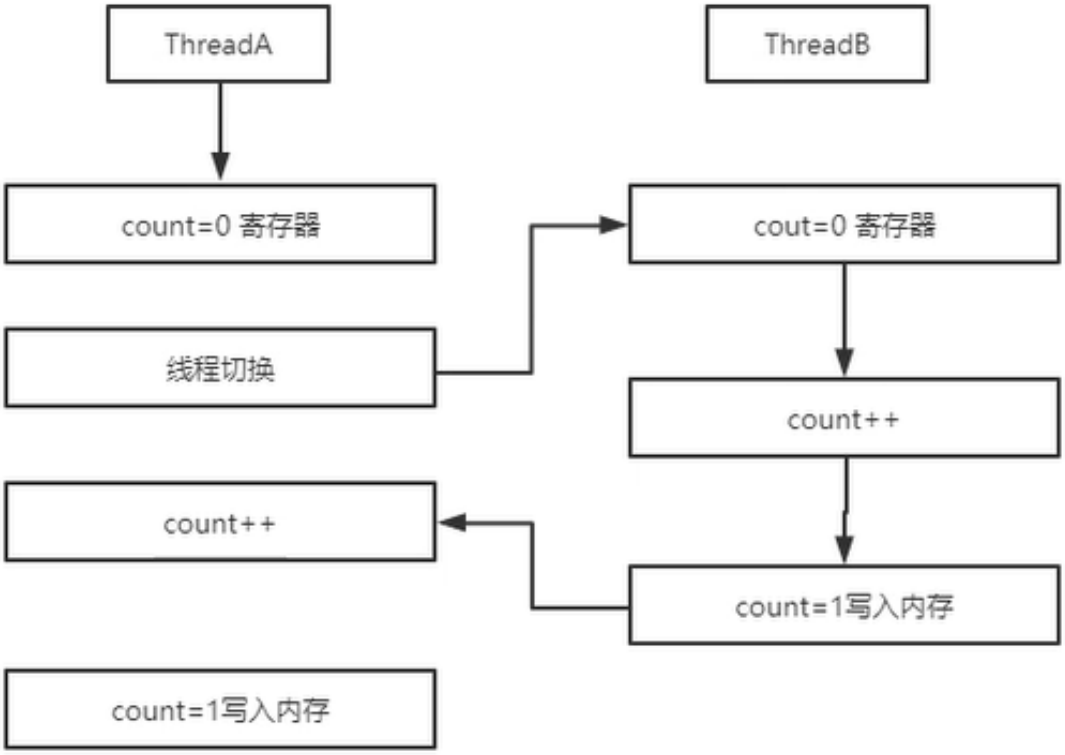
原子性：

通过 javap -v TestDemo.class 查看二进制字节码

```
12: getstatic     #5                  // Field count:I
15: iconst_1
16: iadd
17: putstatic     #5
```

invokestatic：调用静态方法
invokespecial：调用实例构造器方法、私有方法和父类方法
invokevirtual：调用虚方法，运行期动态查找的过程。
invokedynamic：动态调用方法。
invokeinterface:调用接口中的方法，实际上是在运行期决定的，决定到底调用实现该接口的那个对象的特定方法
1.静态方法、私有方法、final 方法、实例构造器、父类方法都是非虚方法，其他方法都是虚方法。

执行过程：



## 2.2 锁 Synchronized

### 2.2.1 互斥锁的本质

共享资源

### 2.2.2 Synchronized 的基本使用

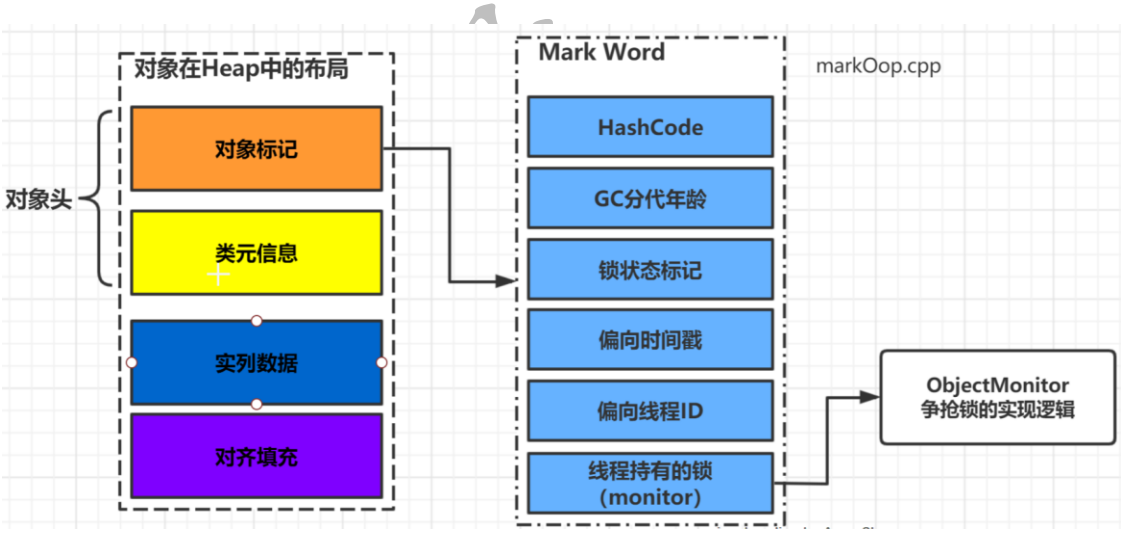
可以修饰在方法层面和代码块层面

synchronized 有三种方式来加锁，不同的修饰类型，代表锁的控制粒度：

1. 修饰实例方法，作用于当前实例加锁，进入同步代码前要获得当前实例的锁
2. 静态方法，作用于当前类对象加锁，进入同步代码前要获得当前类对象的锁
3. 修饰代码块，指定加锁对象，对给定对象加锁，进入同步代码库前要获得给定对象的锁

### 2.2.3 锁的存储（对象头）

在我们最常用的虚拟机 hotspot 中对象在内存中的分布可以分为三个部分：对象头（Header）、实列数据（Instance Data）、对其填充（Padding）




- 通过上述的图示我们可以看到，对象在内存中，包含三个部分， 其中对象头内分为 对象标记与类元信息，在对象标记中主要包含如图所示 hashcode、GC 分代年龄、锁标记状态、偏向锁持有线程 id、线程持有的锁(monitor)等六个内容，这部分数据的长度在 32 位和 64 位的虚拟机中分别为 32bit 和 64bit，在官方将这部分称为 Mark Word 。

- Mark Word 实际是一中可以动态定义的数据结构，这样可以让极小的空间存储尽量多的数据，根据对象的状态复用自己的内存空间，比如在 32 位的虚拟机中，如果对象未被同步锁锁定的状态下， Mark Word 的 32 个比特存储单元中，25 个用于存储哈希码，4 个用于存储 GC 分代年龄，2 个存锁标记位，1 个固定位 0，针对各个状态下的分布可以直观的参看下面的图表

| 锁状态 | 25bit | | 4bit | 1bit | 2bit |
|---|---|---|---|---|---|
| | 23bit | 2bit | | 是否偏向锁 | 锁标志位 |
| 无锁态 | 对象的hashCode | | 分代年龄 | 0 | 01 |
| 轻量级锁 | 指向栈中锁纪录的指针 | | | | 00 |
| 重量级锁 | 指向互斥量(重量级锁)的指针 | | | | 10 |
| GC标记 | 空 | | | | 11 |
| 偏向锁 | 线程ID | Epoch | 分代年龄 | 1 | 01 |

### 2.2.4 打印类的布局

源码在 thread-demo02 项目中

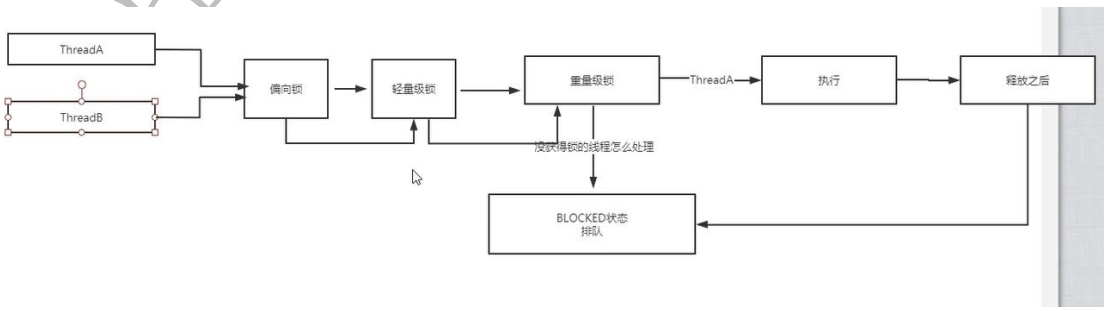
org.openjdk.jol

jol-core

0.10

```
ClassLayout.parseInstance(classLayoutDemo).toPrintable()
```

### 2.2.5 锁升级

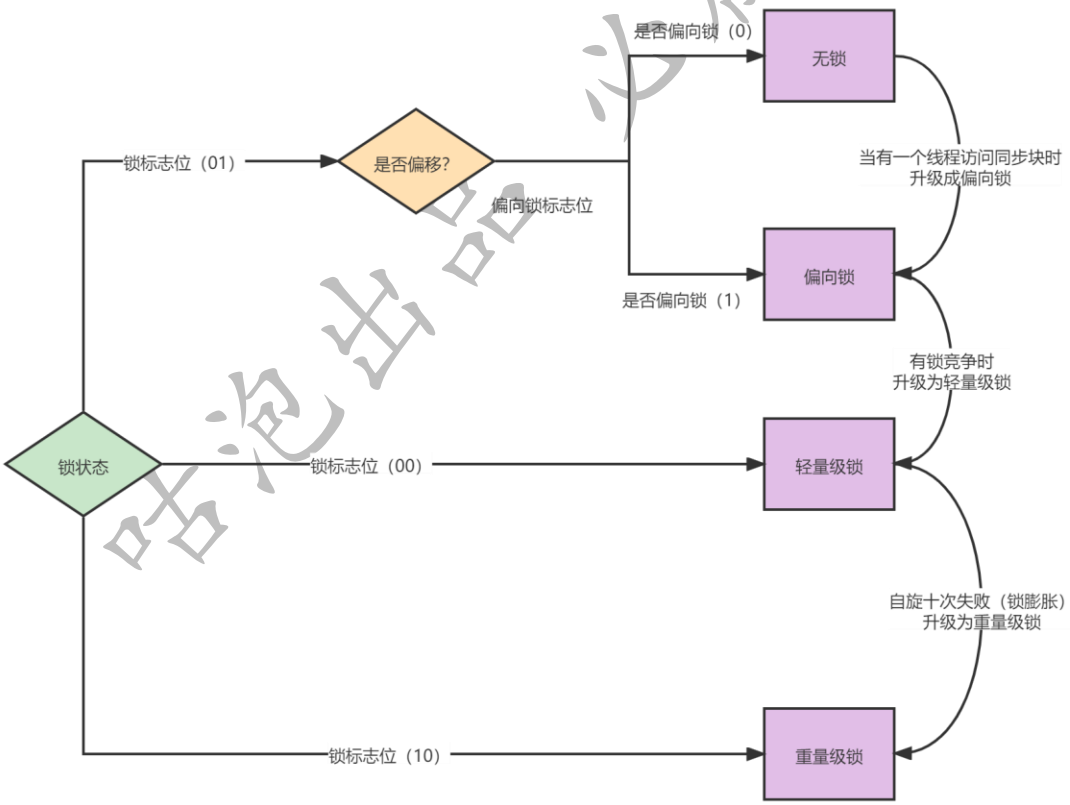



锁的状态总共有四种，级别由低到高依次为：**无锁、偏向锁、轻量级锁、重量级锁**

在 JDK6.0 之前,重量级，1.6 引用锁升级的概念

其实在 JDK 1.6 之前，**synchronized 还是一个重量级锁**，是一个效率比较低下的锁，但是在 JDK 1.6 后，Jvm 为了提高锁的获取与释放效率对（**synchronized** ）进行了优化，引入了 **偏向锁 和 轻量级锁** ，从此以后锁的状态就有了四种（无锁、偏向锁、轻量级锁、重量级锁），**并且四种状态会随着竞争的情况逐渐升级，而且是不可逆的过程，即不可降级，也就是说只能进行锁升级（从低级别到高级别），不能锁降级（高级别到低级别）**，意味着偏向锁升级成轻量级锁后不能降级成偏向锁。这种锁升级却不能降级的策略，目的是为了提高获得锁和释放锁的效率。

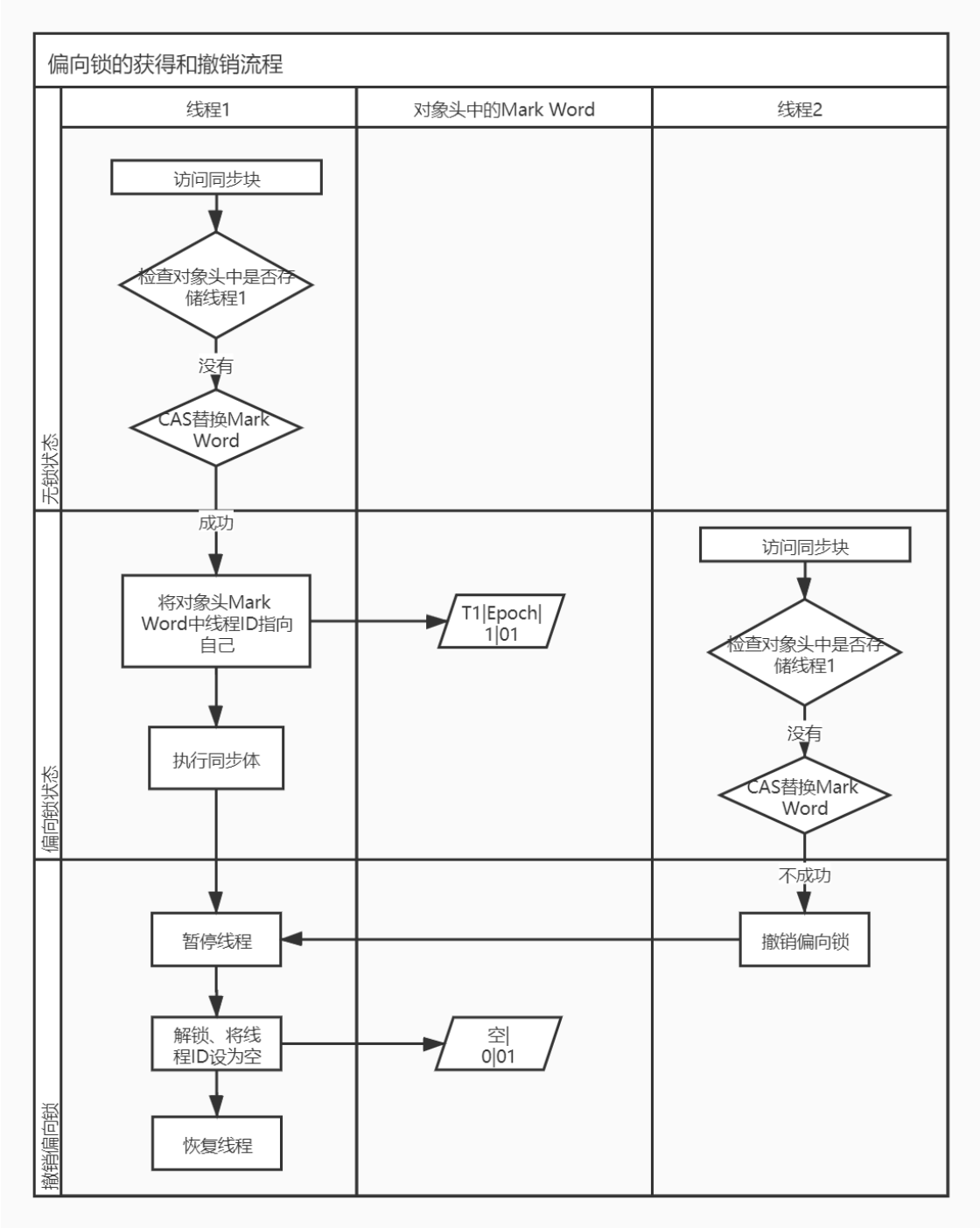
在 synchronized 最初的实现方式是 “**阻塞或唤醒一个 Java 线程需要操作系统切换 CPU 状态来完成，这种状态切换需要耗费处理器时间，如果同步代码块中内容过于简单，这种切换的时间可能比用户代码执行的时间还长**”，这种方式就是 synchronized 实现同步最初的方式，这也是当初开发者诟病的地方，这也是在 JDK6 以前 synchronized 效率低下的原因，JDK6 中为了减少获得锁和释放锁带来的性能消耗，引入了“偏向锁”和“轻量级锁”




https://www.cnblogs.com/mingyueyy/p/13054296.html

### 2.2.5.1 偏向锁

加锁和解锁不需要额外的消耗，和执行非同步方法相比仅存在纳秒级的差距，如果线程间存在锁竞争，会带来额外的锁撤销的消耗，但是只会竞争一次，所以适用在一个线程访问同步代码块，但是这样的业务估计就没有，所以默认也关闭了，因为就算有，也都会上升到轻量级锁

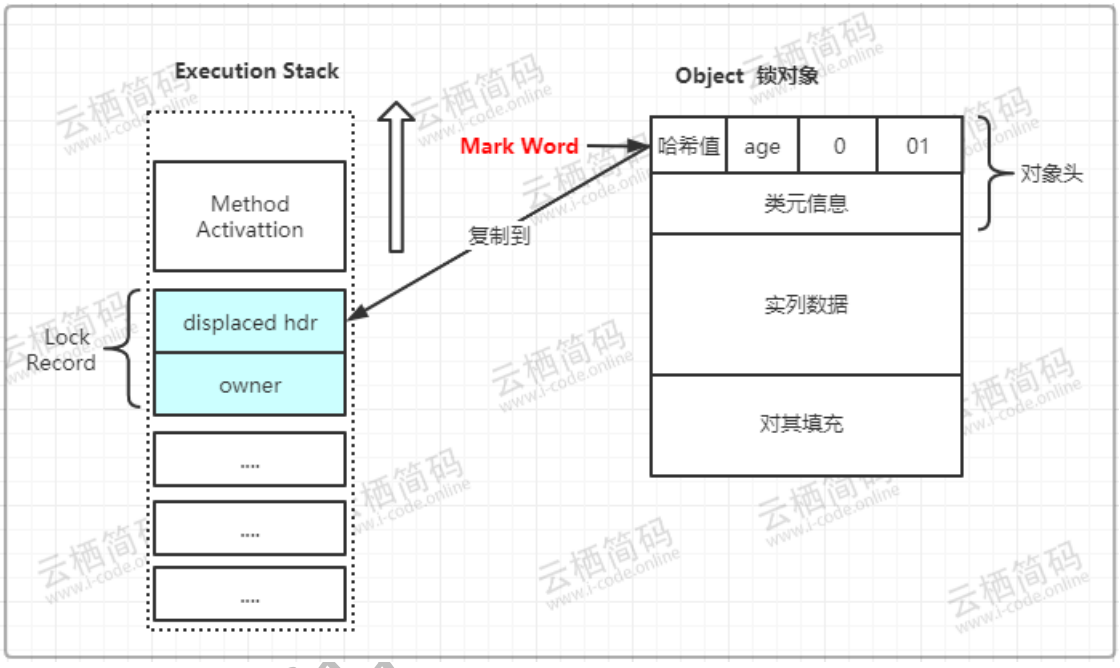


在 java15 中已经默认禁用了偏向锁 http://openjdk.java.net/projects/jdk/15/

### 2.2.5.2 轻量级锁

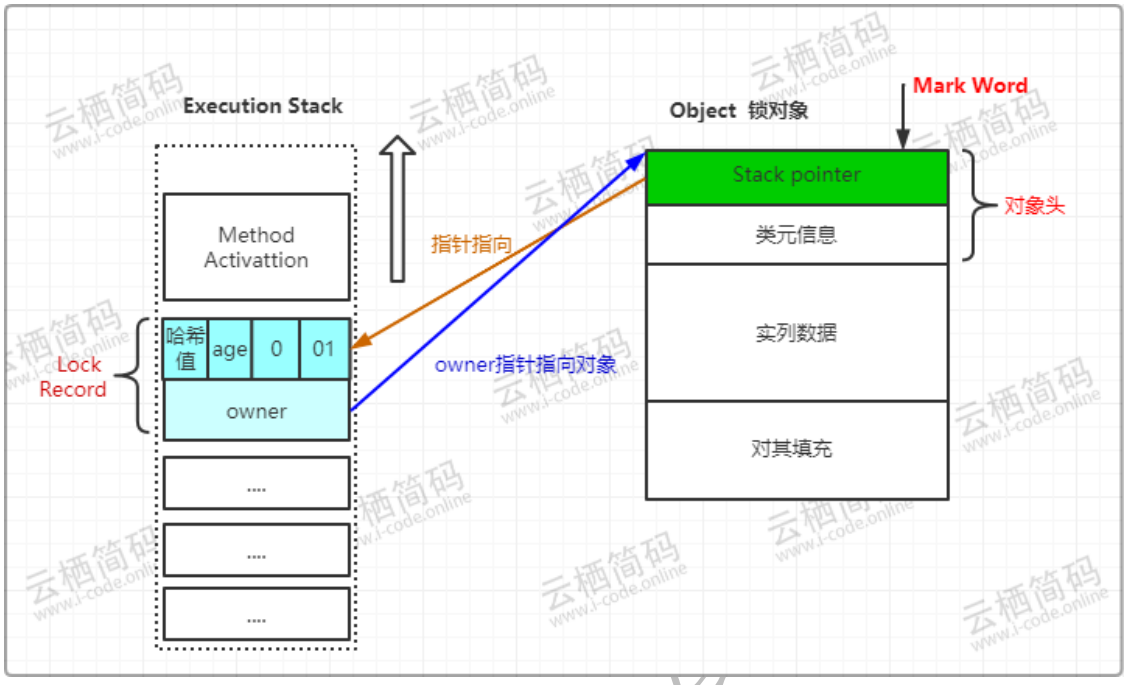
- 当代码执行到同步代码块时，如果同步对象没有被锁定也就是锁标志位为 01 状态，那么虚拟机首先将在当前线程的栈帧中建立一个名为锁记录 Lock Record 的空间

- 这块锁记录空间用来存储锁对象目前的 Mark Word 的拷贝，官方给其加了个 Displaced 的前缀，即 Displaced Mark Word，如下图所示，这是在 CAS 操作之前堆栈与对象的状态



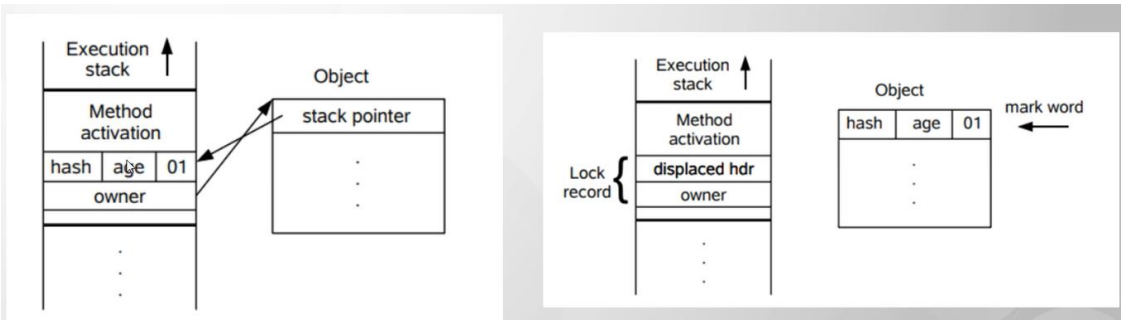

- 当复制结束后虚拟机会通过 CAS 操作尝试把对象的 Mark Word 更新为指向 Lock Record 的指针，如果更新成功则代表该线程拥有了这个对象的锁，并且将 Mark Word 的锁标志位（最后两个比特）转变为“00”，此时表示对象处于

轻量级锁定状态，此时的堆栈与对象头的状态如下：




- 如果上述操作失败了，那说明至少存在一条线程与当前线程竞争获取该对象的锁，虚拟机会首先检查对象的 Mark Word 是否指向当前线程的栈帧，如果是，则说明当前线程已经拥有了这个对象的锁，那么直接进入同步代码块执行即可。否则则说明这个对象已经被其他线程抢占了。
- 如果有超过两条以上的线程争夺同一个锁的情况，那么轻量级锁就不再有效，必须膨胀为重量级锁，锁的标记位也变为“10”，此时 Mark Word 中存储的就是指向重量级锁的指针，等待的线程也必须进入阻塞状态

那么轻量级锁针对于偏向锁的主要不是之前的一次竞争，而是会自旋竞争，所以会消耗 cpu，但是比起后面的重量级锁性能还是快很多，毕竟不会阻塞线程



在 jdk1.6 以前，默认轻量级锁自旋次数是 10 次，如果超过这个次数或自旋线程数超过 CPU 核数的一半，就会升级为重量级锁！当然次数也可以更改

-XX:PreBlockSpin

jdk1.6 以后加入了自适应自旋锁，自旋的次数不再固定，由 jvm 自己控制，由前一次在同一个锁上的自旋时间及锁的拥有者的状态来决定：

- 对于某个锁对象，如果自旋等待刚刚成功获得过锁，并且持有锁的线程正在运行中，那么虚拟机就会认为这次自旋也是很有可能再次成功，进而允许自旋等待持续相对更长时间
- 对于某个锁对象，如果自旋很少成功获得过锁，那在以后尝试获取这个锁时将可能省略掉自旋过程，直接阻塞线程，避免浪费处理器资源。

轻量级锁使用自旋到一定次数后还没有拿到锁，10 次，就会升级为重量级锁，线程阻塞

### 2.2.5.3 重量级锁

- 重量级锁也就是上述几种优化都无效后，膨胀为重量级锁，通过互斥量来实现，我们先来看下面的代码

```
/**
 * @url: i-code.online
 * @author: AnonyStar
 * @time: 2020/10/23 10:36
 */
public class Demo2 {


    public static void main(String[] args) {
        int i =1;
        synchronized (Demo2.class){
            i = 2;
        }
    }
}
```

```
接口
字段
方法
  [0] <init>
  [1] main
    [0] Code
      [0] Line
      [1] Loca
      [2] Stac
属性

属性名索引: cp_info #6 <Code>
属性长度: 132
特有信息
字节码  异常表  杂项
0 iconst_1
1 istore_1
2 ldc #2 <online/icode/threadpool/
4 dup
5 astore_2
6 monitorenter
7 iconst_2
8 istore_1
9 aload_2
10 monitorexit
11 goto 19 (+8)
14 astore_3
15 aload_2
16 monitorexit
17 aload_3
18 athrow
19 return
```

- 上面代码是一个简单使用了 synchronized 的代码，我们通过字节码工具可以看到右侧窗口。我们发现，在同步代码块的前后分别形成了 monitorenter 和 monitorexit 两条指令
- 在 Java 对现中都会有一个 monitor 的监视器，这里的 monitorenter 指令就是去获取一个对象的监视器。而相应的 monitorexit 则表示释放监视器 monitor 的所有权，允许被其他线程来获取
- monitor 是依赖于系统的 MutexLock (互斥锁) 来实现的，当线程阻塞后进入内核态事，就会造成系统在用户态和内核态之间的切换，进而影响性能

只能升级不能降级

#### 2.2.5.4 乐观锁

CAS 比较预期数据和原始数据是否一致，如果一致则修改，否则修改失败。

## 2.3 锁的问题

轻量级锁对象 hashcode 存储在栈帧里面，线程会在栈帧里面开辟一个叫 LockRecord 的空间，然后把对象头中的数据复制到 LockRecord 中，同时对象头会存储该 LockRecord 的地址。

重量级对象 hashcode 存储在 ObjectMonitor

## 2.4 Java 如何实现多线程之间的通讯和协作

1. wait/notify notifyall （通过妻子和丈夫拿车子和钥匙的死锁代码举例）

# 3 探索线程安全性背后的本质

线程安全性问题：

1、原子性问题（上节课已经讲了）

2、有序性问题（简单描述一下多线程的有序性问题）

3、可见性问题（本节课要讲的重点）

## 3.1 一个可见性问题引发的思考

```
public class VolatileDemo01 {
    public static boolean stop = false;
    public static void main(String[] args) throws InterruptedException
{
        Thread thread = new Thread(()->{
           int i=0;
           while(!stop){
               i++;
           }
            System.out.println("rs:"+i);
        });

        thread.start();
        Thread.sleep(1000);
        stop = true;
    }
}
```

咕泡出品 必属精品

这段代码 while 循环不会结束，为什么呢？

因为变量线程不可见

```
//当循环几次之后，JIT 会帮我们自动优化
while(!stop){
    i++;
    System.out.println("rs:"+i);
}
//等价于下面的代码（JIT 自动帮我们优化之后的代码，此时程序已经执行到if 语句里面，在不断的循环while），
if(!stop){
    while(true){
        i++;
    }
}
```

如何解决：

```
public class VolatileDemo02 {
    public static boolean stop = false;
    //public volatile static boolean stop = false;//3、加个volatile 也可以结束
    public static void main(String[] args) throws InterruptedException
{
        Thread thread = new Thread(()->{
           int i=0;
           while(!stop){
               i++;
               try {
                   Thread.sleep(5);
               } catch (InterruptedException e) {
                   e.printStackTrace();
               }//2、睡眠一下也可以结束
//             System.out.println("rs:"+i);//1、打印放在循环里面也可以结束
           }
            System.out.println("rs:"+i);
        });
        thread.start();
        Thread.sleep(1000);
        stop = true;
    }
}
```

咕泡出品 必属精品

volatile 可以禁止 JIT 的自动优化，也可以加个参数 -Djava.compiler=NONE 禁止 JIT 自动优化

## 3.2 print 就可以导致循环结束

这叫做活性失败，它做了一个深度优化

print 里面有两个操作

IO 操作：IO 效率远远低于 CPU 执行效率，所以 IO 会使得线程进入到阻塞状态（等待阻塞，锁阻塞，IO 阻塞），这其实没有证据证明线程进入到阻塞状态之后再次执行会同步内存中的数据，至少我没找到，但是这种情况会破坏 JIT 的深度优化。new File("111");

synchronized 操作：锁释放之后会把缓存中的内容与内存中的内容进行同步。

## 3.3 Thread.sleep()

导致时间片主动释放，释放后线程进入到等待状态，下次获得时间片之后，缓存数据会与内存同步。

## 3.4 硬件层面

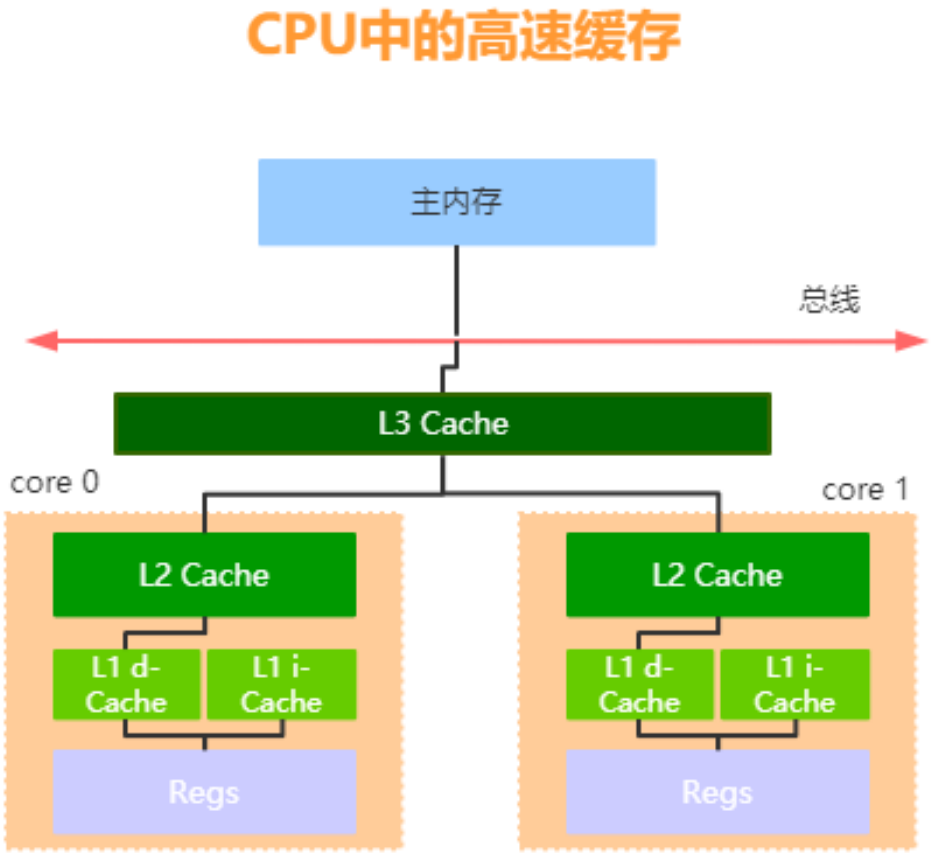
硬件设备的不断升级优化就是为了提升我们计算机的处理性能

CPU 计算速度>>内存 IO 速度>>IO 设备 IO 速度。//木桶原理

- CPU 层面增加了高速缓存（可见性的根本问题）
- 操作系统，进程、线程，CPU 时间片来切换
- 编译器的优化，更合理的利用 CPU 的高速缓存。（比如说线程 A 用到了某个数据，然后会吧这个数据存在高速缓存里面，然后后续线程 B 也用到了这个数据的时候，就会把后续的用到这个缓存数据的指令往前移，这就导致了指令重排）

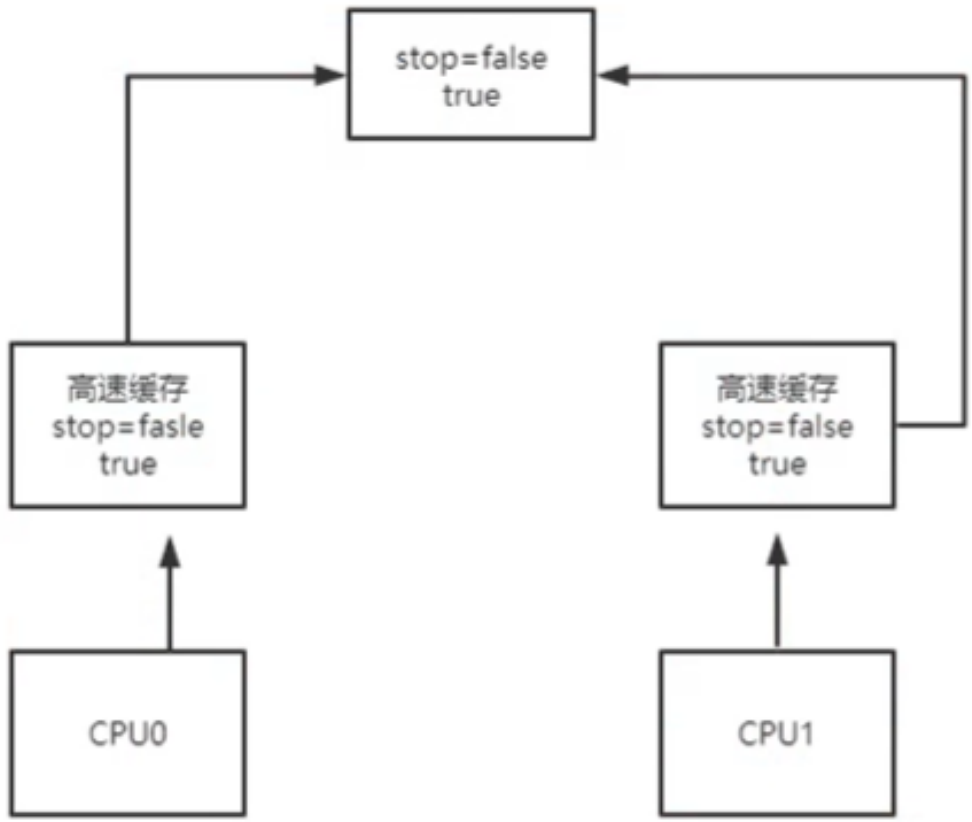
咕泡出品 必属精品

### 3.4.1 CPU 的高速缓存




缓存包含 L1（L1d-数据缓存（存放数据）、L1i 指令缓存（执行数据的指令码））、L2、L3，其中 L3 是 CPU 共享，而 L1 和 L2 是每个 cpu 独占的缓存空间。

这三级缓存都是集成在 CPU 内的缓存结构，他们的作用就是为 CPU 和主内存之间提供一个高速缓冲区，L1 最靠近 CPU 核心；L2 其次；L3 再次。运行速度方面：L1 最快、L2 次快、L3 最慢；容量大小方面：L1 最小、L2 较大、L3 最大。CPU 会先在最快的 L1 中寻找需要的数据，找不到再去找次快的 L2，还找不到再去找 L3，L3 都没有那就只能去内存找了。




因为 L1、L2 缓存是独立的，所以就会导致什么问题？ 可见性问题！！当多个线程并行执行一个共享数据时，都同时在本 cpu 的高速缓存中的时候，最新的数据对其他的 cpu 是不可见的！！所以 CPU 又引入了总线锁与缓存锁

### 3.4.2 总线锁和缓存锁

总线锁，首先得知道总线，其实总线就是负责将各个 CPU 连接到内存！！我们平时买电脑的时候，有个总线频率，比如 1066MHz、1333MHz、1600MHz，频率越高，数据传输量越大！！

那么总线锁可以理解为重量级锁或者悲观锁，当我一个 cpu 在处理数据时，其他 cpu 都是阻塞！！

这样肯定不可取！！所以又继续优化！！就出来了缓存锁

（缓存锁的条件：CPU 的架构是否支持，当前数据是否存在于缓存行，当这两个条件都满足的时候会优先触发缓存所，如果有不满足的话就触发总线锁）

相对于总线锁，降低了锁的力度，那么它怎么去保存缓存的数据一致性？ 缓存锁引入了缓存一致性协议来保证,有 MSI,MESI.MOSI 等等！

### 3.4.3 MESI

最常见的就是 MESI 协议：

1. M(Modify) 被修改 表示共享数据只缓存在当前 CPU 缓存中，并且是被修改状态，也就是缓存的数据和主内存中的数据不一致。
2. E(Exclusive) 独享的 表示缓存的独占状态，数据只缓存在当前 CPU 缓存中，并且没有被修改
3. S(Shared) 共享的 表示数据可能被多个 CPU 缓存，并且各个缓存中的数据和主内存数据一致
4. I(Invalid) 无效的 表示缓存已经失效，不过读或者写都要从主内存和别的缓存读取数据。

演示地址：https://www.scss.tcd.ie/Jeremy.Jones/vivio/caches/MESI.htm

演示，3 个 cpu，~~总线又分为数据总线跟地址总线~~，分为 read 跟 write, wirte 就是要变量+1

1.假如我们 cpu0 操作 writea0,步骤为 cpu0 修改，通过 bus 先从内存拿有没有，并且跟其他 cpu 有个交互，再写入 cpu0,状态为 M，cpu0 跟主内存的数据不一致

2.cpu0 读取 a0,直接从 cpu0 读取，因为在 cpu0 的 cache

3.cpu0write a0,直接操作 cpu0

4.我们 cpu2 读取 a0，我们通过总线 bus，会从其他 cpu 以及主内存交互，从 cpu0 拿到 a0=2，并且把 cpu0=2 的结果同步到主内存，并且 cpu0 跟 cpu1 的数据全部更改为 s 状态，这个时候各个缓存中的数据和主内存数据一致

5.我们在 cpu1 中再 writea0，我们会从主内存，以及其他 cpu 中同步数据到 cpu1，更改为 s，然后再通知其他 cpu 改为 I，cpu1 更改为 E,独享

### 3.4.4 MESI 带来的问题

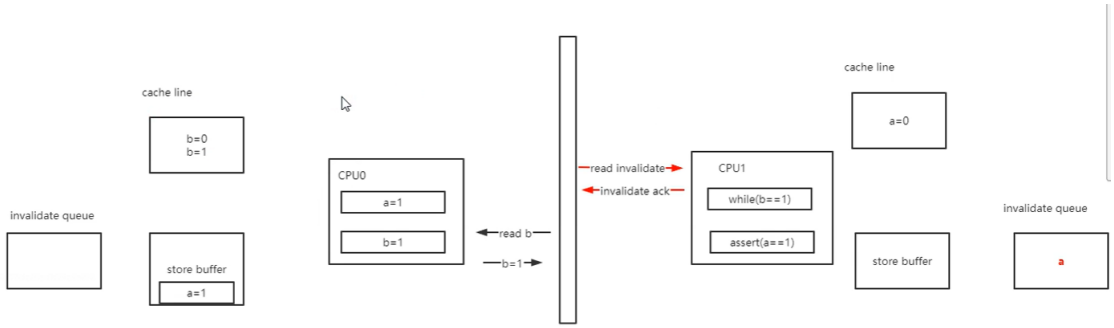
各 CPU 是互相通知工作的，比如现在核 1 想要修改这个变量，首先核 1 会向所有拥有相同缓存的其他核发送一个请求，告诉其他核中的缓存是 I（Invalid）无效的，其他核收到这个信息将自己核中的缓存状态设置为无效之后，返回一个设置完成的消息,1 才能改状态为 M 进行修改！！

陈浩出品 必属精品

这样，CPU1 是不是就一直阻塞没事干，所以，设计者又引入了写缓存（Store Buffer）跟无效化队列（Invalidate Queue）

这 2 个又带来了指令重排问题，我们又引入了内存屏障来解决，针对 Store Buffer 的写屏障，Invalidate Queue 的读屏障

指令重排问题：




```
executeToCPU0(){
      a=1;
    b=1;
}
executeToCPU1(){
    while(b==1){
        assert(a==1)
    }
}
```

### 3.4.5 内存屏障

写屏障（Store Memory Barrier）：强制将 Store Buffer 中的内容写入到缓存中或者将该指令之后的写操作写入 store buffer 直到之前的内容被刷入到缓存中，也被称之为 smp_wmb
读屏障（Load Memory Barrier）：强制将 Invalidate Queue 中的内容处理完毕，也被称之为 smp_rmb

全屏障（Fence Memory Barrier）

## 3.5 软件层面

从 JMM 层面了解可见性

### 3.5.1 什么是 JMM

即 JavaTM 内存模型与线程规范，由 JSR-133 专家组开发

内存模型可以理解为在特定的操作协议下，对特定的内存或者高速缓存进行读写访问的过程抽象描述，不同架构下的物理机拥有不一样的内存模型，Java 虚拟机是一个实现了跨平台的虚拟系统，因此它也有自己的内存模型，即 Java 内存模型（Java Memory Model, JMM）。

因此它不是对物理内存的规范，而是在虚拟机基础上进行的规范从而实现平台一致性，以达到 Java 程序能够“一次编写，到处运行”。

究竟什么是内存模型？

内存模型描述了程序中各个变量（实例域、静态域和数组元素）之间的关系，以及在实际计算机系统中将变量存储到内存和从内存中取出变量这样的底层细节

Java Memory Model(Java 内存模型)， 围绕着在并发过程中如何处理可见性、原子性、有序性这三个特性而建立的模型。

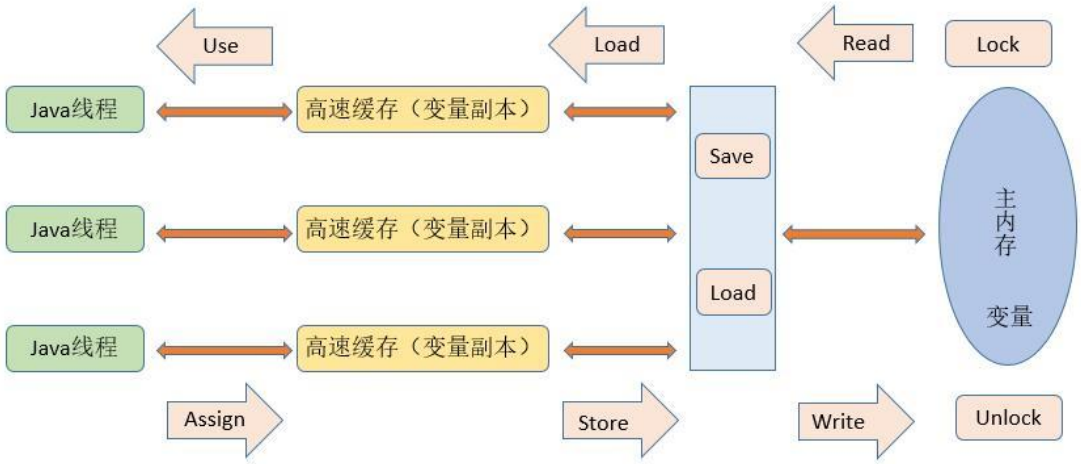
#### 3.5.1.1 JMM 结构规范

JMM 规定了所有的变量都存储在主内存（Main Memory）中。每个线程还有自己的工作内存（Working Memory）,线程的工作内存中保存了该线程使用到的变量的主内存的副本拷贝，线程对变量的所有操作（读取、赋值等）都必须在工作内存中进行，而不能直接读写主内存中的变量（volatile 变量仍然有工作内存的拷贝，但是由于它特殊的操作顺序性规定，所以看起来如同直接在主内存中读写访问一般）。不同的线程之间也无法直接访问对方工作内存中的变量，线程之间值的传递都需要通过主内存来完成。
局部变量（Local variables），方法定义参数（java 语言规范称之为 formal method parameters）和异常处理器参数（exception handler parameters）不会在线程之间共享，它们不会有内存可见性问题，也不受内存模型的影响。

#### 3.5.1.2 主内存和本地内存结构

从抽象的角度来看，JMM 定义了线程和主内存之间的抽象关系：线程之间的共享变量存储在主内存（main memory）中，每个线程都有一个私有的本地内存（local memory），本地内存中存储了该线程以读/写共享变量的副本。本地内存是 JMM 的一个抽象概念，并不真实存在。本地内存它涵盖了缓存，写缓冲区，寄存器以及其他的硬件和编译器优化之后的一个数据存放位置

咕泡出品 必属精品




### 3.5.1.3JMM 的三个特征

Java 内存模型是围绕着并发编程中原子性、可见性、有序性这三个特征来建立的，那我们依次看一下这三个特征：

#### *3.5.1.3.1 原子性（Atomicity）*

一个操作不能被打断，要么全部执行完毕，要么不执行。在这点上有点类似于事务操作，要么全部执行成功，要么回退到执行该操作之前的状态。

i = 2; 赋值给 i ----------操作步骤：1 原子操作
j = i; 读取 i 值 赋值给 j ----------操作步骤：2 非原子操作
i++; 读取 i 值 i 值加 1 赋值给 i ----------操作步骤：3 非原子操作
i = i+1; 读取 i 值 i 值加 1 赋值给 i ----------操作步骤：3 非原子操作

基本类型数据的访问大都是原子操作，long 和 double 类型的变量是 64 位，但是在 32 位 JVM 中，32 位的 JVM 会将 64 位数据的读写操作分为 2 次 32 位的读写操作来进行，这就导致了 long、double 类型的变量在 32 位虚拟机中是非原子操作，数据有可能会被破坏，也就意味着多个线程在并发访问的时候是线程非安全的。

#### *3.5.1.3.2 可见性：*

一个线程对共享变量做了修改之后，其他的线程立即能够看到（感知到）该变量的这种修改（变化）。

Java 内存模型是通过将在工作内存中的变量修改后的值同步到主内存，在读取变量前从主内存刷新最新值到工作内存中，这种依赖主内存的方式来实现可见性的。

无论是普通变量还是 volatile 变量都是如此，区别在于：volatile 的特殊规则保证了 volatile 变量值修改后的新值立刻同步到主内存，每次使用 volatile 变量前立即

从主内存中刷新，因此 volatile 保证了多线程之间的操作变量的可见性，而普通变量则不能保证这一点。

除了 volatile 关键字能实现可见性之外，还有 synchronized,Lock，final 也是可以的。

使用 synchronized 关键字，在同步方法/同步块开始时（Monitor Enter）,使用共享变量时会从主内存中刷新变量值到工作内存中（即从主内存中读取最新值到线程私有的工作内存中），在同步方法/同步块结束时(Monitor Exit),会将工作内存中的变量值同步到主内存中去（即将线程私有的工作内存中的值写入到主内存进行同步）。

使用 Lock 接口的最常用的实现 ReentrantLock(重入锁)来实现可见性：当我们在方法的开始位置执行 lock.lock()方法，这和 synchronized 开始位置（Monitor Enter）有相同的语义，即使用共享变量时会从主内存中刷新变量值到工作内存中（即从主内存中读取最新值到线程私有的工作内存中），在方法的最后 finally 块里执行 lock.unlock()方法，和 synchronized 结束位置（Monitor Exit）有相同的语义,即会将工作内存中的变量值同步到主内存中去（即将线程私有的工作内存中的值写入到主内存进行同步）。

final 关键字的可见性是指：被 final 修饰的变量，在构造函数数一旦初始化完成，并且在构造函数中并没有把“this”的引用传递出去（“this”引用逃逸是很危险的，其他的线程很可能通过该引用访问到只“初始化一半”的对象），那么其他线程就可以看到 final 变量的值。

#### *3.5.1.3.3 有序性：*

对于一个线程的代码而言，我们总是以为代码的执行是从前往后的，依次执行的。这么说不能说完全不对，在单线程程序里，确实会这样执行；但是在多线程并发时，程序的执行就有可能出现乱序。用一句话可以总结为：在本线程内观察，操作都是有序的；如果在一个线程中观察另外一个线程，所有的操作都是无序的。前半句是指“线程内表现为串行语义（WithIn Thread As-if-Serial Semantics）”,后半句是指“指令重排”现象和“工作内存和主内存同步延迟”现象。

一个最经典的例子就是银行汇款问题，一个银行账户存款 100，这时一个人从该账户取 10 元，同时另一个人向该账户汇 10 元，那么余额应该还是 100。那么此时可能发生这种情况，A 线程负责取款，B 线程负责汇款，A 从主内存读到 100，B 从主内存读到 100，A 执行减 10 操作，并将数据刷新到主内存，这时主内存数据 100-10=90，而 B 内存执行加 10 操作，并将数据刷新到主内存，最后主内存数据 100+10=110，显然这是一个严重的问题，我们要保证 A 线程和 B 线程有序执行，先取款后汇款或者先汇款后取款，此为有序性。

尚硅谷出品 版权所属

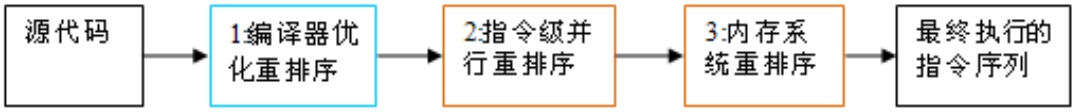
Java 提供了两个关键字 volatile 和 synchronized 来保证多线程之间操作的有序性,volatile 关键字本身通过加入内存屏障来禁止指令的重排序，而 synchronized 关键字通过一个变量在同一时间只允许有一个线程对其进行加锁的规则来实现，

在单线程程序中，不会发生“指令重排”和“工作内存和主内存同步延迟”现象，只在多线程程序中出现。

#### 3.5.1.4 重排序

在执行程序时为了提高性能，编译器和处理器常常会对指令做重排序。重排序分三种类型：

编译器优化的重排序。编译器在不改变单线程程序语义的前提下，可以重新安排语句的执行顺序。
指令级并行的重排序。现代处理器采用了指令级并行技术（Instruction-Level Parallelism， ILP）来将多条指令重叠执行。如果不存在数据依赖性，处理器可以改变语句对应机器指令的执行顺序。
内存系统的重排序。由于处理器使用缓存和读/写缓冲区，这使得加载和存储操作看上去可能是在乱序执行。
从 java 源代码到最终实际执行的指令序列，会分别经历下面三种重排序：
上述的 1 属于编译器重排序，2 和 3 属于处理器重排序。这些重排序都可能会导致多线程程序出现内存可见性问题。对于编译器，JMM 的编译器重排序规则会禁止特定类型的编译器重排序（不是所有的编译器重排序都要禁止）。对于处理器重排序，JMM 的处理器重排序规则会要求 java 编译器在生成指令序列时，插入特定类型的内存屏障（memory barriers，intel 称之为 memory fence）指令，通过内存屏障指令来禁止特定类型的处理器重排序（不是所有的处理器重排序都要禁止）。




JMM 属于语言级的内存模型，它确保在不同的编译器和不同的处理器平台之上，通过禁止特定类型的编译器重排序和处理器重排序，为程序员提供一致的内存可见性保证。

### 3.5.2 Volatile 的原理

#### 3.5.2.1 关键词 synchronized 与 volatile 总结

synchronized 的特点
一个线程执行互斥代码过程如下：

1. 获得同步锁；
2. 清空工作内存；
3. 从主内存拷贝对象副本到工作内存；
4. 执行代码(计算或者输出等)；
5. 刷新主内存数据；
6. 释放同步锁。

所以，**synchronized 既保证了多线程的并发有序性，又保证了多线程的内存可见性和原子性。**

volatile 是第二种 Java 多线程同步的手段，一个变量可以被 volatile 修饰，在这种情况下内存模型确保所有线程可以看到一致的变量值

所以 volatile 可以保证内存可见性，能保证并发有序性。

**可以这么理解：没有 volatile 修饰的共享表量，线程在操作它的时候会先拷贝到缓存中，然后在缓存中修改，然后在同步到内存中。有 volatile 修饰的共享变量，线程在操作它时，直接在内存中进行操作了，不会再拿到缓存中**

1.禁止编译器对代码进行某些优化

2.lock 汇编指令，锁住缓存行，启动内存屏障，禁止指令重排，保证有序性与可见性

当然，除了 volatile 来保证可见性外，我们从 JDK1.5 开始就引用了 happens-before 的概念来保证可见性！！

保证共享数据在多线程环境下可见性的一个关键字 。

通过 javap -v VolatileDemo02.class

```
public static volatile boolean stop;
    descriptor: Z
    flags: ACC_PUBLIC, ACC_STATIC, ACC_VOLATILE
```

#### 3.5.2.2 双检索单例模型中 volatile 的运用

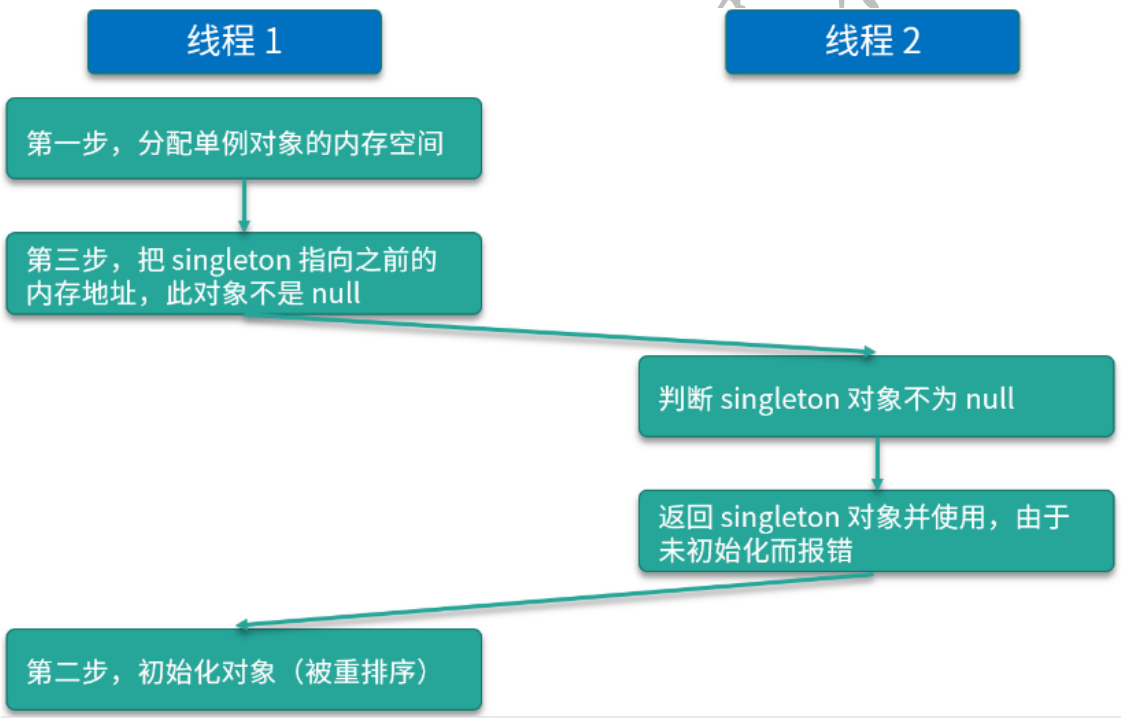
在 java 内存模型中，volatile 关键字作用可以是保证可见性或者禁止指令重排。这里是因为 singleton = new Singleton() ，它并非是一个原子操作，事实上，在 JVM 中上述语句至少做了以下这 3 件事：

咕泡出品 必属精品

- 第一步是给 singleton 分配内存空间；
- 第二步开始调用 Singleton 的构造函数等，来初始化 singleton；
- 第三步，将 singleton 引用指向分配的内存空间（执行完这步 singleton 就不是 null 了）。

这里需要留意一下 1-2-3 的顺序，因为存在指令重排序的优化，也就是说第 2 步和第 3 步的顺序是不能保证的，最终的执行顺序，可能是 1-2-3，也有可能是 1-3-2。

如果是 1-3-2，那么在第 3 步执行完以后，singleton 就不是 null 了，可是这时第 2 步并没有执行，singleton 对象未完成初始化，它的属性的值可能不是我们所预期的值。假设此时线程 2 进入 getInstance 方法，由于 singleton 已经不是 null 了，所以会通过第一重检查并直接返回，但其实这时的 singleton 并没有完成初始化，所以使用这个实例的时候会报错，详细流程如下图所示：




线程 1 首先执行新建实例的第一步，也就是分配单例对象的内存空间，由于线程 1 被重排序，所以执行了新建实例的第三步，也就是把 singleton 指向之前分配出来的内存地址，在这第三步执行之后，singleton 对象便不再是 null。

这时线程 2 进入 getInstance 方法，判断 singleton 对象不是 null，紧接着线程 2 就返回 singleton 对象并使用，由于没有初始化，所以报错了。最后，线程 1 “姗姗来迟”，才开始执行新建实例的第二步——初始化对象，可是这时的初始化已经晚了，因为前面已经报错了。

使用了 volatile 之后，相当于是表明了该字段的更新可能是在其他线程中发生的，因此应确保在读取另一个线程写入的值时，可以顺利执行接下来所需的操作。在 JDK 5 以及后续版本所使用的 JMM 中，在使用了 volatile 后，会一定程度禁止相关语句的重排序，从而避免了上述由于重排序所导致的读取到不完整对象的问题的发生

### 3.5.3 Happens-Before 可见性模型

happens-before 是 JMM 定义的 2 个操作之间的偏序关系：如果操作 A 线性发生于操作 B，则 A 产生的影响能被操作 B 观察到，“影响”包括修改了内存中共享变量的值、发送了消息、调用了方法等。如果两个操作满足 happens-before 原则，那么不需要进行同步操作，JVM 能够保证操作具有顺序性，此时不能够随意的重排序。否则，无法保证顺序性，就能进行指令的重排序。

#### 3.5.3.1 程序顺序规则

一个线程中的每个操作，happens-before 这个线程中的任意后续操作，可以简单认为是 as-if-serial。

as-if-serial 的意思是，不管怎么重排序，单线程的程序的执行结果不能改变。

- 处理器不能对存在依赖关系的操作进行重排序，因为重排序会改变程序的执行结果。
- 对于没有依赖关系的指令，即便是重排序，也不会改变在单线程环境下的执行结果。

具体来看下面这段代码，A 和 B 允许重排序，但是 C 是不允许重排，因为存在依赖关系。根据 as-if-serial 语义，在单线程环境下， 不管怎么重排序，最终执行的结果都不会发生变化。

```
int a=2; //A
int b=2; //B
int c=a*b; //C
```

咕泡出品 必屬精品

#### 3.5.3.2 传递性规则

仍然看下面这段代码，根据程序顺序规则可以知道，这三者之间存在一个 happens-before 关系。

```
int a=2; //A
int b=2; //B
int c=a*b; //C
```

- A happens-before B。
- B happens-before C。
- A happens-before C。

这三个 happens-before 关系，就是根据 happens-before 的传递性推导出来的。很多同学这个时候又有疑惑了，老师，你不是说，A 和 B 之间允许重排序吗？那是不是 A happens-before B 不一定存在，也可能是 B 可以重排序在 A 之前执行呢？

没错，确实是这样，JMM 不要求 A 一定要在 B 之前执行，但是他要求的是前一个操作的执行结果对后一个操作可见。这里操作 A 的执行结果不需要对操作 B 可见，并且重排序操作 A 和操作 B 后的执行结果与 A happens-before B 顺序执行的结果一致，这种情况下，是允许重排序的。

#### 3.5.3.3 volatile 变量规则

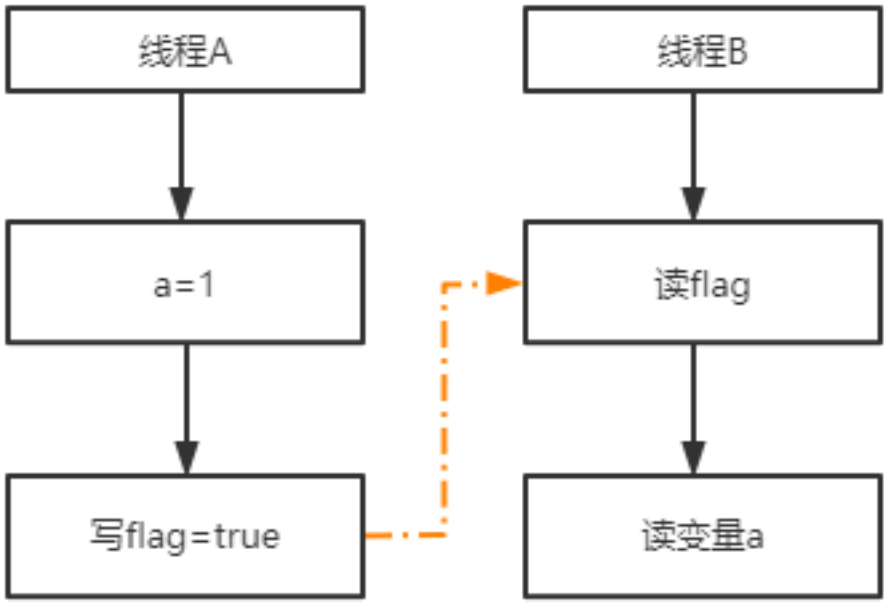
对于 volatile 修饰的变量的写操作，一定 happens-before 后续对于 volatile 变量的读操作，这个是因为 volatile 底层通过内存屏障机制防止了指令重排，这个规则前面已经分析得很透彻了，所以没什么问题，我们再来观察如下代码，基于前面两种规则再结合 volatile 规则来分析下面这个代码的执行顺序，假设两个线程 A 和 B，分别访问 writer 方法和 reader 方法，那么它将会出现以下可见性规则。

```
public class VolatileExample {
    int a=0;
    volatile boolean flag=false;
    public void writer(){
        a=1;            //1
        flag=true;      //2
    }
    public void reader(){
        if(flag){     //3
            int i=a;    //4
        }
    }
}
```

- 1 happens before 2、 3 happens before 4， 这个是程序顺序规则

- 2 happens before 3、 是由 volatile 规则产生的，对一个 volatile 变量的读，总能看到任意线程对这个 volatile 变量的写入。
- 1 happens before 4， 基于传递性规则以及 volatile 的内存屏障策略共同保证。

那么最终结论是，如果在线程 B 执行 reader 方法时，如果 flag 为 true，那么意味着 i=1 成立。




这里有同学可能会有疑问说，老师，你前面讲的程序顺序规则中，在单线程中，如果两个指令之间不存在依赖关系，是允许重排序的，也就是 1 和 2 的顺序可以重排，那么是不是意味着最终 4 输出的结果是 0 呢？

这里也是因为 volatile 修饰的重排序规则的存在，导致 1 和 2 是不允许重排序的，在 volatile 重排序规则表中，如果第一操作是普通变量的读/写，第二个操作是 volatile 的写，那么这两个操作之间不允许重排序。

**表3-5 volatile重排序规则表**

| 是否能重排序 | 第二个操作 | | |
|---|---|---|---|
| 第一个操作 | 普通读 / 写 | volatile 读 | volatile 写 |
| 普通读 / 写 | | | NO |
| volatile 读 | NO | NO | NO |
| volatile 写 | | NO | NO |

#### 3.5.3.4 监视器锁规则

一个线程对于一个锁的释放锁操作，一定 happens-before 与后续线程对这个锁的加锁操作。

```
int x=10;synchronized (this) { // 此处自动加锁
    // x 是共享变量, 初始值 =10
    if (this.x < 12) {
        this.x = 12;
    }
} // 此处自动解锁
```

假设 x 的初始值是 10，线程 A 执行完代码块后，x 的值会变成 12，执行完成之后会释放锁。 线程 B 进入代码块时，能够看到线程 A 对 x 的写操作，也就是 B 线程能够看到 x=12。

#### 3.5.3.5 start 规则

如果线程 A 执行操作 ThreadB.start(),那么线程 A 的 ThreadB.start()之前的操作 happens-before 线程 B 中的任意操作。

```
public class Test{
    public static void main(String[] args){
        final int x[]={0};
        Thread t1 = new Thread(()->{
            // 主线程调用 t1.start() 之前
            // 所有对共享变量的修改，此处皆可见
            // 此例中，x==10
            System.out.println(x[0]);
        });
        // 此处对共享变量 x 修改
        x[0] = 10;
        // 主线程启动子线程
        t1.start();
    }
}
```

#### 3.5.3.6 join 规则

join 规则，如果线程 A 执行操作 ThreadB.join()并成功返回，那么线程 B 中的任意操作 happens-before 于线程 A 从 ThreadB.join()操作成功的返回。

```
public class Test{
    public static void main(String[] args){
        final int[] x = {0};
        Thread t1 = new Thread(()->{
            // 此处对共享变量 x 修改
            x[0] = 100;
        });
```

咕泡出品 必属精品

```java
        // 例如此处对共享变量修改，
        // 则这个修改结果对线程 t1 可见
        // 主线程启动子线程
        t1.start();
        try {
            t1.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        // 子线程所有对共享变量的修改
        // 在主线程调用 t1.join() 之后皆可见
        // 此例中，x==100
        System.out.println(x[0]);
    }
}
```

咕泡出品 必属精品